

Assignment 6

1. Java Application with Jenkins Pipeline

Create a simple Java "Hello World" application.

```
mojorojo@mojorojo-vmware:~/jenkinsjava/newjen/jenkins-java$ cat Hello.java
class Hello{
    public static void main(String args[]){
        System.out.println("Hello, World");
    }
}
```

Push the source code to a GitHub repository.

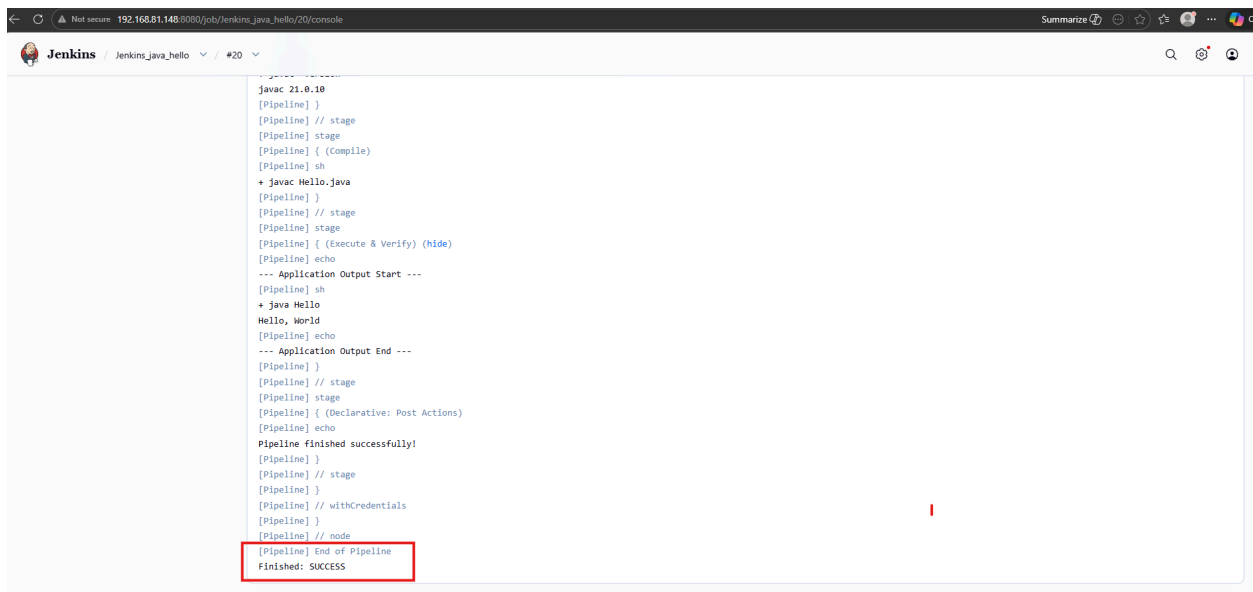
Create a Jenkins Pipeline job to:

Pull the code from GitHub

Compile the Java program

Execute the application

Verify the output in Jenkins console.



The screenshot shows the Jenkins web interface for a job named 'jenkins_java_hello'. The console output is displayed in a monospaced font. The pipeline script is as follows:

```
java: 21.0.10
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Compile)
[Pipeline] sh
+ javac Hello.java
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Execute & Verify) (hide)
[Pipeline] echo
--- Application Output Start ---
[Pipeline] sh
+ java Hello
Hello, World
[Pipeline] echo
--- Application Output End ---
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline finished successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

The final line of the console output, "[Pipeline] End of Pipeline" and "Finished: SUCCESS", is highlighted with a red rectangular box.

2. Docker Image Creation using Jenkins

Create a Dockerfile for the Java application.

```
mojorojo@mojorojo-vmware:~/jenkinsjava/q22$ cat Dockerfile
FROM eclipse-temurin:17-jdk-alpine
WORKDIR /app
COPY Hello.java .
RUN javac Hello.java
CMD ["java", "Hello"]
```

Update the Jenkins Pipeline to:

Build a Docker image from the application

Run the image as a Docker container locally on the Jenkins server

Verify the container output.

```

mojorojo@mojorojo-vmware:~/jenkinsjava/q22$ cat Jenkins
pipeline {
  agent any

  stages {
    stage(' Compile Java') {
      steps {
        sh 'javac Hello.java'
      }
    }

    stage(' Build Docker Image') {
      steps {
        sh 'docker build -t my-app .'
      }
    }

    stage(' Run & Verify') {
      steps {
        sh 'docker run --name my-app-containernew2 my-app'
      }
    }
  }
}

```

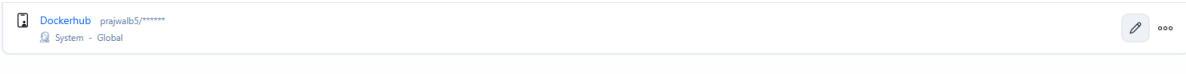
```

#9 exporting to image
#9 exporting layers 0.1s done
#9 exporting manifest sha256:c7fbc3a06b7bcd4cfd3f65e55c6f3113f3537da374d56cb083259c913e974dd 0.0s done
#9 exporting config sha256:dbbabc069616db6c8947e9a3da1c1f4ca8530dc9ee81600b4ba9e5cfa284a61 0.0s done
#9 exporting attestation manifest sha256:655a9334c84052127fda0f978b3bd3f6a7798a9afdb2b69b072f0850ab4f99af 0.0s done
#9 exporting manifest list sha256:89ee9a2628f99bbe51ae380e23e1b5cd91fd080169c0c9fd153d7de6248096b done
#9 naming to docker.io/library/my-app:latest done
#9 unpacking to docker.io/library/my-app:latest 0.0s done
#9 DONE 0.2s
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { ( Run & Verify)
[Pipeline] sh
+ docker run --name my-app-containernew2 my-app
Hello, World
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

1. Push Docker Image to DockerHub

Configure DockerHub credentials in Jenkins.



Extend the Jenkins Pipeline to:
Tag the Docker image properly

```
pipeline {
  agent any

  stages {
    stage('Run Container') {
      steps {
        sh 'docker rm -f ngi || true'
        sh 'docker run -d --name ngi -p 8000:80 nginx:latest'
      }
    }

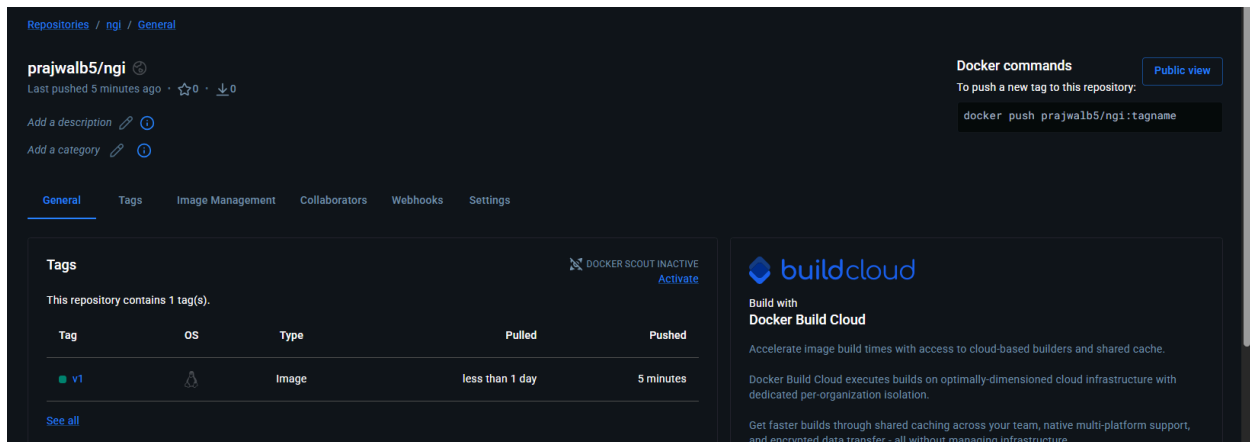
    stage('Tag & Push') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'Dockerhub',
                                          passwordVariable: 'DOCKER_PASS',
                                          usernameVariable: 'DOCKER_USER')]) {

          sh "echo ${DOCKER_PASS} | docker login -u ${DOCKER_USER} --password-stdin"

          // Tag and Push
          sh 'docker tag nginx:latest prajwalb5/ngi:v1'
          sh 'docker push prajwalb5/ngi:v1'

          sh 'docker logout'
        }
      }
    }
  }
}
```

Push the image to DockerHub repository



Verify the image is available in DockerHub

```
4677c2a9a3d4: Mounted from library/nginx
ff048f1f2159: Mounted from library/nginx
3531af2bc2a9: Mounted from library/nginx
v1: digest: sha256:296499281873ebe2636681c6957776d29a31858d93c3e39b654b5aade87ed70a size: 2290

Info -> Not all multiplatform-content is present and only the available single-platform image was pushed
[31msha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc[0m -> [32msha256:296499281873ebe2636681c6957776d29a31858d93c3e39b654b5aade87ed70a[0m

[Pipeline] sh
+ docker logout
Removing login credentials for https://index.docker.io/v1/
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

4. Deploy Application to Kubernetes using Jenkins

Create a Kubernetes deployment and service YAML file.

Extend the Jenkins Pipeline to:

Pull the Docker image from DockerHub

Deploy the application as a Kubernetes Pod

Verify the pod is running successfully in the cluster

```
Code Blame 33 lines (30 loc) · 1.04 KB
1 pipeline {
2   agent any
3
4   stages {
5     stage ('pull image') {
6       steps {
7         sh 'docker pull nginx:latest'
8       }
9     }
10    stage ('run pod') {
11      steps {
12        sh 'kubectl apply -f pod.yaml'
13        sh 'kubectl get pods'
14      }
15    }
16    stage ('Running pod inside a deployment '){
17      steps{
18        sh 'kubectl create deployment k8snew --image nginx:latest'
19      }
20    }
21    stage ('Exposing as a service'){
22      steps{
23        sh 'kubectl expose deployment k8snew --type=NodePort --port=80'
24        sh 'kubectl get pods'
25
26        sh 'kubectl delete pod k8spodnew --ignore-not-found'
27        sh 'kubectl delete service k8snew --ignore-not-found'
28        //sh 'kubectl delete deployment k8s1--ignore-not-found'
29      }
30    }
31  }
32 }
33 }
```

```

k8s-79bcc48fbb-7kspk    1/1    Running    0        6m23s
k8s1-57678996bc-xl2s4  1/1    Running    0        2m59s
k8spod                  1/1    Running    0        61s
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Running pod inside a deployment )
[Pipeline] sh
+ kubectl create deployment k8snew --image nginx:latest
deployment.apps/k8snew created
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Exposing as a service)
[Pipeline] sh
+ kubectl expose deployment k8snew --type=NodePort --port=80
service/k8snew exposed
[Pipeline] sh
+ kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
k8s-79bcc48fbb-7kspk              1/1     Running             0          6m24s
k8s1-57678996bc-xl2s4            1/1     Running             0          3m
k8snew-6c95977fc6-bbw7d          0/1     ContainerCreating   0          1s
k8spod                            1/1     Running             0          62s
[Pipeline] sh
+ kubectl delete pod k8spodnew --ignore-not-found
[Pipeline] sh
+ kubectl delete service k8snew --ignore-not-found
service "k8snew" deleted from default namespace
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```